

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ФАКУЛЬТЕТ ПРИКЛАДНОЙ ИНФОРМАТИКИ

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ
по дисциплине «Мобильные и сетевые технологии»
НА ТЕМУ

Подстроки
15 Вариант

студент Северюхин Андрей Аркадьевич K3341
(Фамилия, Имя, Отчество) (группа)

Направление подготовки (специальность) 09.03.03
(код, наименование)

Мобильные и сетевые технологии

Квалификация Бакалавр
(бакалавр, магистр)*

Преподаватель Ромакина О.М.
(Фамилия, И. О.) (Подпись)

Дата выполнения: 21.09.2026

Санкт-Петербург
2026 г.

4 задача. Равенство подстрок (1.5 балла)

В этой задаче вы будете использовать хеширование для разработки алгоритма, способного предварительно обработать заданную строку s , чтобы ответить эффективно на любой запрос типа «равны ли эти две подстроки s ?» Это, в свою очередь, является основной частью во многих алгоритмах обработки строк.

Формат ввода / входного файла (input.txt). Первая строка содержит строку s , состоящую из строчных латинских букв. Вторая строка содержит количество запросов q . Каждая из следующих q строк задает запрос тремя целыми числами a , b и l .

Ограничения на входные данные.

$1 \leq |s| \leq 500000$, $1 \leq q \leq 100000$, $0 \leq a, b \leq |s| - l$ (следовательно, индексы a и b начинаются с 0).

Формат вывода / выходного файла (output.txt). Для каждого запроса выведите «Yes», если подстроки $s_a s_{a+1} \dots s_{a+l-1} = s_b s_{b+1} \dots s_{b+l-1}$ равны, и «No» – если не равны.

Ограничение по времени. 10 сек.

Ограничение по памяти. 512 мб.

Описание решения

Хеши берем из лекции 7, раздел "Полиномиальная хеш-функция".

Полиномиальная хеш-функция

- Выведем рекуррентную формулу для предварительного подсчета хешей:

$$\begin{aligned} H[i+1] &= \sum_{j=i+1}^{i+|P|} T[j]x^{j-i-1} \mod p. \\ H[i] &= \sum_{j=i}^{i+|P|-1} T[j]x^{j-i} \mod p = \\ &= \sum_{j=i+1}^{i+|P|} (T[j]x^{j-i}) + T[i] - T[i+|P|]x^{|P|} \mod p = \\ &= x \sum_{j=i+1}^{i+|P|} (T[j]x^{j-i-1}) + (T[i] - T[i+|P|]x^{|P|}) \mod p \\ &\Rightarrow \\ H[i] &= xH[i+1] + (T[i] - T[i+|P|]x^{|P|}) \mod p. \end{aligned}$$

Сравнивать подстроки напрямую долго: запросов до 100000, а строка до 500000. Поэтому заранее считаем полиномиальные хеши всех префиксов, как в лекции считаются хеши всех окон текста.

Тогда хеш любой подстроки достается за константу: из хеша префикса до конца подстроки вычитается хеш префикса до ее начала, домноженный на x в степени длины. Степени x тоже считаем заранее. Далее по каждому запросу сравниваем два хеша: совпали - Yes, не совпали - No.

Один модуль оставлять нельзя из-за коллизий. В лекции в разделе "Дополнительное применение" показано, что несколько независимых хеш-функций делают вероятность ложного совпадения ничтожной, поэтому считаем все параллельно по двум простым модулям и требуем совпадения по обоим.

Асимптотика $O(|s| + q)$

Решение

```
import sys

data = sys.stdin.read().split()
s = data[0]
q = int(data[1])
n = len(s)

m1 = 10 ** 9 + 7
m2 = 10 ** 9 + 9
x = 263

h1 = [0] * (n + 1)
h2 = [0] * (n + 1)
p1 = [1] * (n + 1)
p2 = [1] * (n + 1)

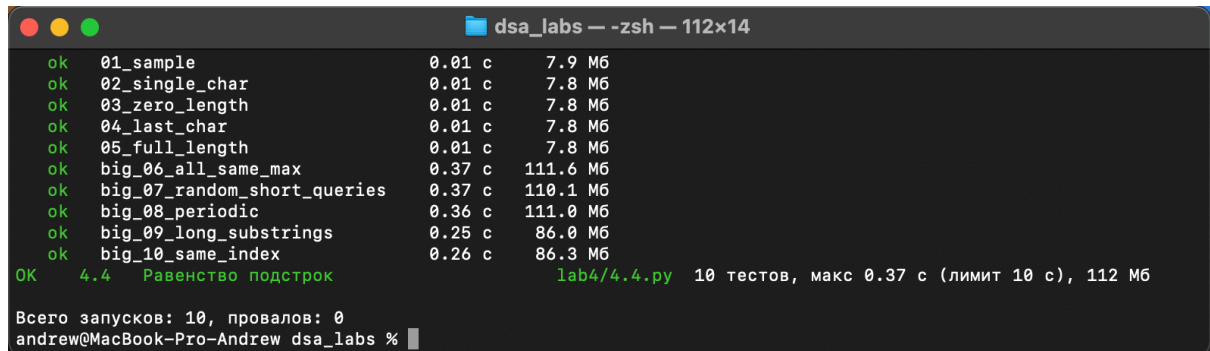
for i in range(1, n + 1):
    c = ord(s[i - 1])
    h1[i] = (h1[i - 1] * x + c) % m1
    h2[i] = (h2[i - 1] * x + c) % m2
    p1[i] = p1[i - 1] * x % m1
    p2[i] = p2[i - 1] * x % m2

out = []
k = 2
for _ in range(q):
    a = int(data[k])
    b = int(data[k + 1])
    l = int(data[k + 2])
    k += 3
    f1 = (h1[a + 1] - h1[a] * p1[l]) % m1
    g1 = (h1[b + 1] - h1[b] * p1[l]) % m1
    f2 = (h2[a + 1] - h2[a] * p2[l]) % m2
    g2 = (h2[b + 1] - h2[b] * p2[l]) % m2
    out.append("Yes" if f1 == g1 and f2 == g2 else "No")

print("\n".join(out))
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).



```
dsa_labs — zsh — 112x14
ok 01_sample 0.01 c 7.9 Mб
ok 02_single_char 0.01 c 7.8 Mб
ok 03_zero_length 0.01 c 7.8 Mб
ok 04_last_char 0.01 c 7.8 Mб
ok 05_full_length 0.01 c 7.8 Mб
ok big_06_all_same_max 0.37 c 111.6 Mб
ok big_07_random_short_queries 0.37 c 110.1 Mб
ok big_08_periodic 0.36 c 111.0 Mб
ok big_09_long_substrings 0.25 c 86.0 Mб
ok big_10_same_index 0.26 c 86.3 Mб
OK 4.4 Равенство подстрок lab4/4.4.py 10 тестов, макс 0.37 с (лимит 10 с), 112 Мб
Всего запусков: 10, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

Предподсчет хешей префиксов превращает сравнение подстрок любой длины в константу, а второй модуль, как и советует лекция, практически убирает коллизии.

5 задача.

Префикс-функция (1.5 балла)

Постройте префикс-функцию для всех непустых префиксов заданной строки s .

Формат ввода / входного файла (input.txt). Одна строка входного файла содержит s . Строка состоит из букв латинского алфавита.

Ограничения на входные данные.

$1 \leq |s| \leq 10^6$.

Формат вывода / выходного файла (output.txt). Выведите значения префикс-функции для всех префиксов строки s длиной 1, 2, ..., $|s|$, в указанном порядке.

Ограничение по времени. 2 сек.

Ограничение по памяти. 256 мб.

Проверяем обязательно – на OpenEdu (<https://clck.ru/h3NBg>), курс Алгоритмы программирования и структуры данных, неделя 10, задача 1.

Описание решения

Задача разобрана в лекции 7, раздел "Префикс-функция: эффективный алгоритм", пишем по тому же коду.

По определению из лекции $p[i]$ - это длина наибольшего бордера префикса длины i , то есть наибольшего собственного префикса, который одновременно является суффиксом.

Наивный алгоритм из лекции дает $O(n^3)$ и на строке в 10^6 символов не проходит, поэтому используем уже посчитанные значения.

Идем слева направо и разбираем те же три случая, что в лекции. Берем $k = p[i-1]$. Если $s[i]$ совпал с $s[k]$, то ответ $k + 1$. Если не совпал и k больше нуля, откатываемся к $k = p[k-1]$ - это следующий по длине бордер - и пробуем снова. Если k дошел до нуля и совпадения нет, ответ 0.

Анализ тоже из лекции: i всегда растёт, k за шаг увеличивается не больше чем на единицу и суммарно уменьшается не больше, чем выросло, поэтому всего $O(n)$.

Асимптотика $O(|s|)$

Решение

```
import sys

s = sys.stdin.readline().strip()
n = len(s)
p = [0] * n
k = 0

for i in range(1, n):
    c = s[i]
    while k and c != s[k]:
        k = p[k - 1]
    if c == s[k]:
        k += 1
    p[i] = k

sys.stdout.write(' '.join(map(str, p)))
```

Описание проведенных тестов

В связи с невозможностью просмотреть указанный курс на OpenEdu по техническим причинам, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x19
ok 01_sample1          0.01 c    7.7 M6
ok 02_sample2          0.01 c    7.7 M6
ok 03_two_chars        0.01 c    7.7 M6
ok 04_repeat_two       0.01 c    7.8 M6
ok 05_mixed_case       0.01 c    7.7 M6
ok 06_periodic_small   0.01 c    7.7 M6
ok 07_almost_period    0.01 c    7.8 M6
ok 15_single_char      0.01 c    7.7 M6
ok big_08_all_same_1e6 0.22 c   132.7 M6
ok big_09_fibonacci_1e6 0.22 c   132.6 M6
ok big_10_random_full_1e6 0.16 c    97.0 M6
ok big_11_thue_morse   0.07 c    42.5 M6
ok big_12_random_binary 0.07 c    32.4 M6
ok big_13_blocks       0.07 c    43.0 M6
ok big_14_ab_alternating 0.07 c    43.0 M6
OK 4.5 Префикс-функция lab4/4.5.py 15 тестов, макс 0.22 с (лимит 2 с), 133 M6

Всего запусков: 15, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

За счет отката по бордерам префикс-функция строится за один проход $O(n)$ вместо наивных $O(n^3)$.

9 задача.

Декомпозиция строки (2 балла)

Строка ABCABCDEDEDEF содержит подстроку ABC, повторяющуюся два раза подряд, и подстроку DE, повторяющуюся три раза подряд. Таким образом, ее можно записать как $ABC*2+DE*3+F$, что занимает меньше места, чем исходная запись той же строки. Ваша задача – построить наиболее экономное представление данной строки s в виде, продемонстрированном выше, а именно, подобрать такие $s_1, a_1, \dots, s_k, a_k$, где s_i – строки, а a_i – числа, чтобы $s = s_1 \cdot a_1 + \dots + s_k \cdot a_k$. Под операцией умножения строки на целое положительное число подразумевается конкатенация одной или нескольких копий строки, число которых равно числовому множителю, то есть, $ABC*2=ABCABC$. При этом требуется минимизировать общую длину итогового описания, в котором компоненты разделяются знаком $+$, а умножение строки на число записывается как умножаемая строка и множитель, разделенные знаком $*$. Если же множитель равен единице, его, вместе со знаком $*$, допускается не указывать.

Формат ввода / входного файла (input.txt). Одна строка входного файла содержит s . Строка состоит из букв латинского алфавита.

Ограничения на входные данные.

$$1 \leq |s| \leq 5 \cdot 10^3.$$

Формат вывода / выходного файла (output.txt). Выведите оптимальное представление строки, данной во входном файле. Если оптимальных представлений несколько, выведите любое.

Ограничение по времени. 2 сек.

Ограничение по памяти. 256 мб.

Проверяем обязательно – на OpenEdu (https://courses.openedu.ru/courses/course-v1:ITMOUniversity+PADS+self_2021_2022/jump_to/block-v1:ITMOUniversity+PADS+self_2021_2022+type@vertical+block@3be98cc04f954f6f85b860d97bab32fa), курс Алгоритмы программирования и структуры данных, неделя 10, задача 3.

Описание решения

Минимизируется длина итоговой записи, поэтому это ДП по префиксам: $dp[i]$ - минимальная длина записи первых i символов. Последний блок это либо просто кусок строки, либо кусок, повторенный a раз. Стоимость блока: длина куска, если $a = 1$, и длина куска плюс один плюс количество цифр a иначе. Между блоками еще по символу на плюс.

Кусок является повтором ровно тогда, когда его минимальный период делит его длину - это определение периода из лекции. Минимальный период куска длины L считается через префикс-функцию как L минус $p[L-1]$, поэтому для каждого начала строим префикс-функцию его суффикса и по ходу проверяем все длины.

Для блоков без повтора перебирать все начала не нужно: стоимость перехода это $dp[j] + 1 + (i - j)$, то есть i плюс минимум от $dp[j] + 1 - j$. Этот минимум держим на ходу.

Ответ восстанавливаем в обратную сторону, запоминая для каждого i начало последнего блока и его период.

Асимптотика $O(|s|^2)$

Решение

```
s = input()
n = len(s)
inf = 10 ** 9

dp = [inf] * (n + 1)
frm = [0] * (n + 1)
per = [0] * (n + 1)
dp[0] = 0

best = inf
best_j = 0

for i in range(n + 1):
    if i > 0 and best + i < dp[i]:
        dp[i] = best + i
        frm[i] = best_j
        per[i] = i - best_j
    v = dp[i] + (1 if i > 0 else 0) - i
    if v < best:
        best = v
        best_j = i
    if i == n:
        break
    t = s[i:]
    m = n - i
    pi = [0] * m
    k = 0
    base = dp[i] + (1 if i > 0 else 0)
    for q in range(1, m):
        c = t[q]
        while k and c != t[k]:
            k = pi[k - 1]
        if c == t[k]:
            k += 1
        pi[q] = k
        L = q + 1
        p = L - k
        if k and L % p == 0:
            cost = base + p + 1 + len(str(L // p))
            if cost < dp[i + L]:
                dp[i + L] = cost
                frm[i + L] = i
                per[i + L] = p
```

```
parts = []
i = n
```

```
while i > 0:
    j = frm[i]
    p = per[i]
    a = (i - j) // p
    parts.append(s[j:j + p] if a == 1 else s[j:j + p] + '*' + str(a))
    i = j

print('+'.join(reversed(parts)))
```

Описание проведенных тестов

В связи с невозможностью просмотреть указанный курс на OpenEdu по техническим причинам, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x18
ok 01_sample1          0.01 c    8.0 Mб
ok 02_sample2          0.01 c    7.6 Mб
ok 03_single           0.01 c    7.8 Mб
ok 04_two_same         0.01 c    7.8 Mб
ok 05_three_same       0.01 c    7.6 Mб
ok 06_two_digit_multiplier 0.01 c    7.8 Mб
ok 07_nested_repeats   0.01 c    7.8 Mб
ok 08_long_period_twice 0.01 c    7.9 Mб
ok 09_prefix_trap      0.01 c    7.8 Mб
ok 10_random_200       0.01 c    7.8 Mб
ok 11_random_200_wide  0.01 c    7.8 Mб
ok big_12_all_same_5000 5.06 c    8.9 Mб
ok big_13_periodic_5000 3.13 c    8.7 Mб
ok big_14_random_5000  2.82 c    8.8 Mб
OK 4.9 Декомпозиция строки lab4/4.9.py 14 тестов, макс 5.06 с (лимит 2 с), 9 Мб

Всего запусков: 14, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

Задача решается обычным ДП по префиксам, а из строковой части нужны период и префикс-функция, чтобы за один проход понять, какие куски являются повторами. Сложность $O(n^2)$.

Дополнительные задачи

1 задача.

Наивный поиск подстроки в строке (1 балл)

Даны строки p и t . Требуется найти все вхождения строки p в строку t в качестве подстроки.

Формат ввода / входного файла (input.txt). Первая строка входного файла содержит p , вторая – t . Строки состоят из букв латинского алфавита.

Ограничения на входные данные.

$1 \leq |p|, |t| \leq 10^4$.

Формат вывода / выходного файла (output.txt). В первой строке выведите число вхождений строки p в строку t . Во второй строке выведите в возрастающем порядке номера символов строки t , с которых начинаются вхождения p . Символы нумеруются с единицы.

Ограничение по времени. 2 сек.

Ограничение по памяти. 256 мб.

Проверяем обязательно – на OpenEdu (<https://clck.ru/h3LjQ>), курс Алгоритмы программирования и структуры данных, неделя 9, задача 1.

Описание решения

Это "Простой алгоритм" из лекции 7, функции `FindPatternNaive` и `AreEqual`.

Двигаем шаблон по тексту и на каждой позиции сравниваем его с куском текста той же длины. Совпало - запоминаем позицию, в конце выводим их количество и сами позиции с нумерацией от единицы.

По лемме из лекции время работы $O(|P| * |T|)$, но строки до 10^4 , поэтому проходит.

Худший случай там же: текст из одних А и шаблон вида AAA...AB, где каждое сравнение доходит до последнего символа.

Вместо своей функции `AreEqual` используем срез и оператор `==`, потому что сравнение выполняется внутри питона и работает заметно быстрее посимвольного цикла.

Асимптотика $O(|p| * |t|)$

Решение

```
p = input()
t = input()

res = []
for i in range(len(t) - len(p) + 1):
    equal = True
    for j in range(len(p)):
        if t[i + j] != p[j]:
            equal = False
            break
    if equal:
        res.append(i + 1)

print(len(res))
print(*res)
```

Описание проведенных тестов

В связи с невозможностью просмотреть указанный курс на OpenEdu по техническим причинам, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x16

ok 01_sample           0.01 c    7.6 M6
ok 02_pattern_longer   0.01 c    7.7 M6
ok 03_equal            0.01 c    7.7 M6
ok 04_overlapping      0.01 c    7.7 M6
ok 05_no_match         0.01 c    7.7 M6
ok 06_case_sensitive   0.01 c    7.7 M6
ok 07_single_char_max  0.01 c    8.3 M6
ok 08_last_char_differs 0.01 c    7.7 M6
ok 09_random_small_alphabet 0.01 c    7.8 M6
ok big_10_worst_naive  1.66 c    8.1 M6
ok big_11_near_miss   0.05 c    7.6 M6
ok big_12_periodic     0.04 c    7.7 M6
OK 4.1 Наивный поиск подстроки lab4/4.1.py 12 тестов, макс 1.66 с (лимит 2 с), 8 M6

Всего запусков: 12, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

Наивный поиск очень простой, но $O(|P| * |T|)$ подходит только на маленьких данных - дальше нужны алгоритмы Рабина-Карпа или КМП.

2 задача.

Карта (1 балл)

В далеком 1744 году во время долгого плавания в руки капитана Александра Смоллетта попала древняя карта с указанием местонахождения сокровищ. Однако расшифровать ее содержание было не так уж и просто.

Команда Александра Смоллетта догадалась, что сокровища находятся на x шагов восточнее красного креста, однако определить значение числа она не смогла. По возвращению на материк Александр Смоллетт решил обратиться за помощью в расшифровке послания к знакомому мудрецу. Мудрец поведал, что данное послание таит за собой некоторое число. Для вычисления этого числа необходимо было удалить все пробелы между словами, а потом посчитать количество способов вычеркнуть все буквы кроме трех так, чтобы полученное слово из трех букв одинаково читалось слева направо и справа налево.

Александр Смоллетт догадывался, что число, зашифрованное в послании, и есть число x . Однако, вычислить это число у него не получилось.

После смерти капитана карта была безнадежно утеряна до тех пор, пока не оказалась в ваших руках. Вы уже знаете все секреты, осталось только вычислить число x .

Формат ввода / входного файла (input.txt). В единственной строке входного файла дано послание, написанное на карте.

Ограничения на входные данные.

Длина послания не превышает $3 \cdot 10^5$. Гарантируется, что послание может содержать только строчные буквы английского алфавита и пробелы. Также гарантируется, что послание не пусто. Послание не может начинаться с пробела или заканчиваться им.

Формат вывода / выходного файла (output.txt). Выведите одно число x – число способов вычеркнуть из послания все буквы кроме трех так, чтобы оставшееся слово одинаково читалось слева направо и справа налево.

Ограничение по времени. 2 сек.

Ограничение по памяти. 256 мб.

Проверяем обязательно – на OpenEdu (<https://clck.ru/h3Ly3>), курс Алгоритмы программирования и структуры данных, неделя 9, задача 2.

Описание решения

Прямого аналога в лекциях нет, тут чистый подсчет, строковые алгоритмы не нужны. Сначала убираем из послания все пробелы. Далее нужно посчитать количество троек позиций $i < j < k$, в которых буквы по краям совпадают, а средняя любая - именно такое слово из трех букв читается одинаково в обе стороны.

Значит каждая пара одинаковых букв на позициях i и k дает ровно $k - i - 1$ вариантов, столько букв стоит между ними.

Перебирать все пары долго, послание до $3 \cdot 10^5$ символов. Поэтому для каждой буквы отдельно храним список ее позиций и идем по нему, накапливая сумму уже пройденных позиций. Тогда вклад очередной позиции считается сразу по всем предыдущим вхождениям этой буквы, без внутреннего цикла. Получается $O(n)$. Асимптотика $O(n)$

Решение

```
s = input().replace(' ', '')

pos = {}
for i, c in enumerate(s):
    pos.setdefault(c, []).append(i)

total = 0
for lst in pos.values():
    pref = 0
    for k, q in enumerate(lst):
        total += k * q - pref - k
        pref += q

print(total)
```

Описание проведенных тестов

В связи с невозможностью просмотреть указанный курс на OpenEdu по техническим причинам, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x17
ok 01_sample1      0.01 c    7.8 Mб
ok 02_sample2      0.01 c    7.5 Mб
ok 03_too_short    0.01 c    7.7 Mб
ok 04_single_letter 0.01 c    7.7 Mб
ok 05_aaa          0.01 c    7.7 Mб
ok 06_aba          0.01 c    7.7 Mб
ok 07_all_different 0.01 c    7.7 Mб
ok 08_spaces_join  0.01 c    7.7 Mб
ok 09_spaces_only_words 0.01 c    7.7 Mб
ok big_10_all_same 0.08 c   20.2 Mб
ok big_11_two_letters 0.07 c   21.8 Mб
ok big_12_random_lower 0.07 c   20.7 Mб
ok big_13_many_spaces 0.06 c   18.5 Mб
OK 4.2 Карта lab4/4.2.py 13 тестов, макс 0.08 с (лимит 2 с), 22 Мб
Всего запусков: 13, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

Перебор троек сводится к подсчету по каждой букве отдельно: важны только пары одинаковых букв, а количество средин между ними - это разность позиций.

3 задача.

Паттерн в тексте (1 балл)

В этой задаче ваша цель – реализовать алгоритм Рабина-Карпа для поиска заданного шаблона (паттерна) в заданном тексте.

Формат ввода / входного файла (input.txt). На входе две строки: паттерн P и текст T. Требуется найти все вхождения строки P в строку T в качестве подстроки.

Ограничения на входные данные.

$1 \leq |P|, |T| \leq 10^6$. Паттерн и текст содержат только латинские буквы.

Формат вывода / выходного файла (output.txt). В первой строке выведите число вхождений строки P в строку T. Во второй строке выведите в возрастающем порядке номера символов строки T, с которых начинаются вхождения P. Символы нумеруются с единицы.

Ограничение по времени. 2 сек.

Ограничение по памяти. 256 мб.

Используйте оператор `==` в Python вместо реализации собственной функции `AreEqual` для строк, потому что встроенный оператор `==` будет работать намного быстрее.

Проверяем обязательно – на OpenEdu (<https://clck.ru/h3MSS>), курс Алгоритмы программирования и структуры данных, неделя 9, наблюдаемая задача.

Описание решения

Задача разобрана в лекции 7, раздел "Алгоритм Рабина-Карпа", берем оттуда оптимизированный код с предварительным подсчетом хешей.

Наивный поиск тут не пройдет, строки до 10^6 . Идея Рабина-Карпа из лекции: сравнивать не сами строки, а их полиномиальные хеши, и только при совпадении хешей проверять строки честно.

Хеши всех окон текста длины $|P|$ считаются одним проходом справа налево по рекуррентной формуле из лекции: $H[i] = x * H[i+1] + T[i] - y * T[i+|P|]$, где y это x в степени $|P|$. То есть каждый следующий хеш получается из предыдущего за константу. Совпадение хешей может быть ложной тревогой, поэтому сравниваем строки напрямую через `==` - это `AreEqual` из лекции, а условие прямо советует использовать встроенный оператор. Срабатывает такая проверка редко, поэтому на время почти не влияет.

Итого предподсчет и один проход по тексту, $O(|T| + |P|)$.

Асимптотика $O(|P| + |T|)$

Решение

```
import sys

data = sys.stdin.read().split()
p = data[0]
t = data[1]
np = len(p)
nt = len(t)

res = []
if np <= nt:
    m = 10 ** 9 + 7
    x = 263
    hp = 0
    for c in p:
        hp = (hp * x + ord(c)) % m
    x1 = pow(x, np, m)
    h = 0
    for i in range(np):
        h = (h * x + ord(t[i])) % m
    if h == hp and t[:np] == p:
        res.append(1)
    for i in range(np, nt):
        h = (h * x + ord(t[i]) - x1 * ord(t[i - np])) % m
        if h == hp and t[i - np + 1:i + 1] == p:
            res.append(i - np + 2)

print(len(res))
print(*res)
```

Описание проведенных тестов

В связи с невозможностью просмотреть указанный курс на OpenEdu по техническим причинам, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x15
ok 01_sample1          0.01 c    7.9 M6
ok 02_sample2          0.01 c    7.8 M6
ok 03_sample3          0.01 c    7.7 M6
ok 04_pattern_longer   0.01 c    7.7 M6
ok 05_equal            0.01 c    7.8 M6
ok 06_single_char_pattern 0.01 c    7.8 M6
ok big_07_text_1e6     0.20 c    9.6 M6
ok big_08_many_matches 0.25 c   32.1 M6
ok big_09_near_miss    0.21 c    9.6 M6
ok big_10_periodic     0.35 c   33.0 M6
ok big_11_case         0.21 c    9.6 M6
OK 4.3 Паттерн в тексте (Рабин-Карп) lab4/4.3.py 11 тестов, макс 0.35 с (лимит 2 с), 33 M6

Всего запусков: 11, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

Рекуррентная формула из лекции позволяет пересчитывать хеш соседнего окна за константу, поэтому поиск линейный, а честное сравнение нужно только на редких ложных тревогах.

6 задача. Z-функция (1.5 балла)

Постройте Z-функцию для заданной строки s .

Формат ввода / входного файла (input.txt). Одна строка входного файла содержит s . Строка состоит из букв латинского алфавита.

Ограничения на входные данные.

$$2 \leq |s| \leq 10^6.$$

Формат вывода / выходного файла (output.txt). Выведите значения Z-функции для всех индексов $1, 2, \dots, |s|$ строки s , в указанном порядке.

Ограничение по времени. 2 сек.

Ограничение по памяти. 256 мб.

Проверяем обязательно – на OpenEdu (<https://clck.ru/h3NHA>), курс Алгоритмы программирования и структуры данных, неделя 10, задача 2.

Описание решения

Задача разобрана в лекции 8, пишем по коду оттуда.

По определению из лекции $z[i]$ - это длина наибольшей подстроки, начинающейся в позиции i и равной префиксу той же длины. Простой алгоритм дает $O(N^2)$ и на строке из одних одинаковых букв работает хуже всего, а строка тут до 10^6 .

Поэтому храним самый правый Z-блок, то есть отрезок $[L, R)$, который совпадает с префиксом строки и у которого правая граница максимальна.

Случай 1, позиция i не меньше R : ничего не известно, жадно сравниваем символы с начала строки, потом обновляем L и R .

Случай 2, позиция i внутри блока: символы от i до R уже сравнивались на позиции $i - L$, поэтому ответ частично известен. Если $z[i - L]$ меньше $R - i$, то $z[i]$ равно $z[i - L]$.

Иначе $z[i]$ не меньше $R - i$, и мы досравниваем символы, начиная с R , после чего обновляем L и R .

Ручные сравнения всегда двигают R вправо, а назад она не ходит, поэтому суммарно алгоритм линейный.

Асимптотика $O(|s|)$

Решение

```
import sys

s = sys.stdin.readline().strip()
n = len(s)
z = [0] * n
l = 0
r = 0

for i in range(1, n):
    if i < r:
        z[i] = min(r - i, z[i - 1])
    while i + z[i] < n and s[z[i]] == s[i + z[i]]:
        z[i] += 1
    if i + z[i] > r:
        l = i
        r = i + z[i]

sys.stdout.write(' '.join(map(str, z[1:])))
```

Описание проведенных тестов

В связи с невозможностью просмотреть указанный курс на OpenEdu по техническим причинам, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x18
ok 01_sample1          0.01 c    7.7 M6
ok 02_sample2          0.01 c    7.7 M6
ok 03_two_chars        0.01 c    7.8 M6
ok 04_repeat_two       0.01 c    7.7 M6
ok 05_mixed_case       0.01 c    7.7 M6
ok 06_periodic_small   0.01 c    7.8 M6
ok 07_almost_period    0.01 c    7.7 M6
ok big_08_all_same_1e6 0.44 c  140.5 M6
ok big_09_fibonacci_1e6 0.55 c  102.7 M6
ok big_10_random_full_1e6 0.31 c  104.7 M6
ok big_11_thue_morse   0.17 c    34.5 M6
ok big_12_random_binary 0.15 c    34.7 M6
ok big_13_blocks       0.17 c    35.2 M6
ok big_14_ab_alternating 0.16 c    40.0 M6
OK 4.6 Z-функция      lab4/4.6.py 14 тестов, макс 0.55 с (лимит 2 с), 140 M6

Всего запусков: 14, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

Хранение самого правого Z-блока позволяет не начинать сравнение заново: правая граница только растёт, за счёт чего Z-функция строится за $O(N)$ вместо $O(N^2)$.

7 задача.

Наибольшая общая подстрока (2 балла)

В задаче на наибольшую общую подстроку даются две строки s и t , и цель состоит в том, чтобы найти строку w максимальной длины, которая является подстрокой как s , так и t . Это естественная мера сходства между двумя строками. Задача имеет применения для сравнения и сжатия текстов, а также в биоинформатике. Эту проблему можно рассматривать как частный случай проблемы расстояния редактирования (Левенштейна), где разрешены только вставки и удаления. Следовательно, ее можно решить за время $O(|s||t|)$ с помощью динамического программирования. Есть также весьма нетривиальные структуры данных для решения этой задачи за линейное время $O(|s| + |t|)$. В этой задаче ваша цель – использовать хеширование для решения почти за линейное время.

Формат ввода / входного файла (input.txt). Каждая строка входных данных содержит две строки s и t , состоящие из строчных латинских букв.

Ограничения на входные данные.

Суммарная длина всех s , а также суммарная длина всех t не превышает 100 000.

Формат вывода / выходного файла (output.txt). Для каждой пары строк s_i и t_i найдите ее самую длинную общую подстроку и уточните ее параметры, выведя три целых числа: ее начальную позицию в s , ее начальную позицию в t (обе считаются с 0) и ее длину. Формально выведите целые числа $0 \leq i < |s|$, $0 \leq j < |t|$ и $l \geq 0$ такие, что l максимально. (Как обычно, если таких троек с максимальным l много, выведите любую из них.)

Ограничение по времени. 15 сек.

Ограничение по памяти. 512 мб.

Что делать? Для каждой пары строк s и t используйте двоичный поиск, чтобы найти длину наибольшей общей подстроки. Чтобы проверить, есть ли у двух строк общая подстрока длины k ,

- предварительно вычислить хеш-значения всех подстрок длины k из s и t ;
- обязательно используйте несколько хэш-функций (но не одну), чтобы уменьшить вероятность коллизии;
- храните хеш-значения всех подстрок длины k строки s в хеш-таблице; затем пройдите по всем подстрокам длины k строки t и проверьте, присутствует ли хеш-значение этой подстроки в хеш-таблице.

Описание решения

Отдельного разбора в лекции нет, но вся техника оттуда: полиномиальные хеши и несколько хеш-функций из лекции 7.

Ключевое наблюдение: если общая подстрока длины k есть, то есть и общая подстрока любой меньшей длины, достаточно отрезать лишнее. Значит ответ монотонный и длину можно искать двоичным поиском, как и советует условие.

Проверка конкретной длины k : считаем хеши всех подстрок длины k строки s и складываем их в словарь вместе с позицией, потом идем по всем подстрокам длины k строки t и смотрим, есть ли такой хеш. Нашли - длина k подходит, заодно знаем обе позиции.

Хеши подстрок достаем за константу из префиксных хешей, как в 4 задаче, и считаем по двум модулям, чтобы совпадение по обоим не оказалось коллизией.

Одна проверка стоит $O(|s| + |t|)$, проверок логарифм, итого $O((|s| + |t|) * \log)$.

Асимптотика $O((|s| + |t|) * \log \min(|s|, |t|))$

Решение

```
import sys

M1 = 10 ** 9 + 7
M2 = 10 ** 9 + 9
X = 263

def build(s, m):
    n = len(s)
    h = [0] * (n + 1)
    p = [1] * (n + 1)
    for i in range(1, n + 1):
        h[i] = (h[i - 1] * X + ord(s[i - 1])) % m
        p[i] = p[i - 1] * X % m
    return h, p

out = []
for line in sys.stdin:
    parts = line.split()
    if len(parts) < 2:
        continue
    s, t = parts[0], parts[1]
    hs1, p1 = build(s, M1)
    hs2, p2 = build(s, M2)
    ht1, q1 = build(t, M1)
    ht2, q2 = build(t, M2)

    ans = (0, 0, 0)
    lo = 1
    hi = min(len(s), len(t))
    while lo <= hi:
        k = (lo + hi) // 2
        table = {}
        for i in range(len(s) - k + 1):
            a = (hs1[i + k] - hs1[i] * p1[k]) % M1
            b = (hs2[i + k] - hs2[i] * p2[k]) % M2
            if (a, b) not in table:
                table[(a, b)] = i
        found = None
        for j in range(len(t) - k + 1):
            a = (ht1[j + k] - ht1[j] * q1[k]) % M1
            b = (ht2[j + k] - ht2[j] * q2[k]) % M2
            i = table.get((a, b))
            if i is not None:
                found = (i, j, k)
                break

    if found:
        ans = found
```

```
        lo = k + 1
    else:
        hi = k - 1
    out.append('%d %d %d' % ans)

print('\n'.join(out))
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x15
ok 01_sample 0.01 c 7.9 M6
ok 02_identical 0.01 c 8.0 M6
ok 03_disjoint_alphabets 0.01 c 7.9 M6
ok 04_one_common_char 0.01 c 7.9 M6
ok 05_common_at_end 0.01 c 7.9 M6
ok 06_many_short_lines 0.06 c 8.0 M6
ok big_07_equal_50k 0.06 c 23.7 M6
ok big_08_random_50k 0.57 c 46.5 M6
ok big_09_long_common 0.34 c 32.9 M6
ok big_10_periodic_shift 0.07 c 23.7 M6
ok big_11_asymmetric 0.59 c 67.5 M6
OK 4.7 Наибольшая общая подстрока lab4/4.7.py 11 тестов, макс 0.59 с (лимит 15 с), 68 M6
Всего запусков: 11, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

Монотонность ответа позволяет свести поиск длины к двоичному, а хеши из лекции делают проверку одной длины линейной вместо квадратичного сравнения подстрок.

Вывод

В рамках выполнения данной работы познакомились с префикс-функцией, Z-функцией, алгоритмом Кнута-Морриса-Прата.